

NeuroGS-Codec: Training Objective

1 Algorithm: Training a 3D Gaussian Implicit Volume (NeuroGS-Codec)

Inputs and Parameters

- Ground-truth volume $V(\mathbf{x})$ on a discrete voxel grid; normalized to $[0, 1]$.
- Coordinate domain $\mathbf{x} \in \Omega \subset \mathbb{R}^3$, using normalized coordinates in $[-1, 1]^3$.
- Number of Gaussians: N (initialized to N_0 , grows via densification).
- Hyperparameters: $\kappa, \lambda, \alpha, \beta_S, \beta_{Sm}, \beta_O, \beta_{TV}, \beta_{SSIM}, \beta_{Edge}, \beta_G$.
- Quantization steps $\Delta_\mu, \Delta_\ell, \Delta_q, \Delta_a, \Delta_b$ (for rate term).

1.1 1. Prepare Training Data and Coordinates

1. **Load and normalize the volume:**

$$V \leftarrow \frac{V - \min(V)}{\max(V) - \min(V) + \epsilon}.$$

2. **Construct a coordinate grid:** for each voxel index (z, y, x) map to a continuous coordinate

$$\mathbf{x}_{zyx} = (x', y', z') \in [-1, 1]^3.$$

3. **Construct a neurite/edge importance map** $m(\mathbf{x}) \in [0, 1]$ (DoG + gradient magnitude) to emphasize tubular/edge geometry during training.

1.2 2. Construct the Implicit Function (Gaussian Mixture Field)

1. **Define the implicit function** $f(\mathbf{x})$ as a weighted sum of Gaussian basis functions:

$$f(\mathbf{x}) = \sum_{i=1}^N a_i G_i(\mathbf{x}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) + b.$$

2. **Define each anisotropic Gaussian basis** as:

$$G_i(\mathbf{x}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right).$$

3. **Parameterize the covariance** for stable optimization:

$$\boldsymbol{\Sigma}_i = R(\mathbf{q}_i) \text{diag}(\mathbf{s}_i^2) R(\mathbf{q}_i)^\top, \quad \mathbf{s}_i = \exp(\boldsymbol{\ell}_i),$$

where $R(\mathbf{q}_i) \in SO(3)$ is a rotation from a unit quaternion $\mathbf{q}_i$ and $\boldsymbol{\ell}_i$ are log-scales.

1.3 3. Define the Loss Function (Distortion + Rate)

1. **Sample M training voxels** $\{(\mathbf{x}_k, v_k, m_k)\}_{k=1}^M$, where $v_k = V(\mathbf{x}_k)$ and $m_k = m(\mathbf{x}_k)$.

2. **Forward prediction:**

$$\hat{v}_k = f(\mathbf{x}_k).$$

3. **Distortion (geometry-weighted Charbonnier):**

$$\mathcal{L}_D = \frac{1}{M} \sum_{k=1}^M \underbrace{(1 + \kappa m_k)}_{\text{geometry weight}} \underbrace{\sqrt{(\hat{v}_k - v_k)^2 + \epsilon^2}}_{\text{Charbonnier}}.$$

4. **Rate (entropy proxy for quantized parameters):**

Quantize parameters using scalar step sizes Δ :

$$\hat{\theta} = \text{round}(\theta/\Delta),$$

and estimate expected bits with a Laplace model:

$$\mathcal{L}_R = \mathcal{R}(\hat{\mu}) + \mathcal{R}(\hat{\ell}) + \mathcal{R}(\hat{\mathbf{q}}) + \mathcal{R}(\hat{\mathbf{a}}) + \mathcal{R}(\hat{b}).$$

1.4 4. Regularization Terms (Proposal-style)

1. **4.1 Weight Sparsity (encourage fewer active Gaussians):**

$$\mathcal{L}_S = \frac{1}{N} \sum_{i=1}^N |a_i|.$$

2. **4.2 Overlap Regularization (discourage redundant Gaussians):**

For sampled pairs (i, j) , define

$$d_{ij}^2 = (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j)^\top (\boldsymbol{\Sigma}_i + \boldsymbol{\Sigma}_j + \epsilon I)^{-1} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j), \quad \text{overlap}_{ij} = \exp(-\frac{1}{2}d_{ij}^2),$$

and penalize expected overlap:

$$\mathcal{L}_O = \mathbb{E}_{(i,j)}[\text{overlap}_{ij}].$$

3. **4.3 Smoothness Regularization (two levels):**

- *Field smoothness (closest to “smooth transitions”):*

$$\mathcal{L}_G = \mathbb{E}_{\mathbf{x} \sim \Omega} [\|\nabla_{\mathbf{x}} f(\mathbf{x})\|_2^2].$$

- *Parameter conditioning (covariance stability):*

$$\mathcal{L}_{\text{Sm}} = \text{penalty to keep scales bounded and avoid extreme anisotropy}.$$

4. **4.4 Patch TV (piecewise smooth geometry):**

$$\mathcal{L}_{\text{TV}} = \mathbb{E}_{\mathcal{P}} [\|\nabla f\|_1].$$

5. **4.5 Patch SSIM (structure preservation):**

$$\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(f(\mathcal{P}), V(\mathcal{P})).$$

6. **4.6 Edge-aware reconstruction (boundary alignment):**

$$\mathcal{L}_{\text{Edge}} = \mathbb{E}_{\mathbf{x} \in \mathcal{P}} [w_{\text{edge}}(\mathbf{x}) |f(\mathbf{x}) - V(\mathbf{x})|].$$

7. **4.7 (Optional) Topology-biased patch loss (neurite continuity):**

$$\mathcal{L}_T = \text{TV}(\sigma(\gamma(f - \tau))) + c_h \|\sigma(\gamma(f - \tau)) - \text{LPF}(\sigma(\gamma(f - \tau)))\|_1.$$

1.5 5. Total Objective

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{D}} + \lambda \mathcal{L}_{\text{R}} + \alpha \mathcal{L}_{\text{T}} + \beta_{\text{TV}} \mathcal{L}_{\text{TV}} + \beta_{\text{SSIM}} \mathcal{L}_{\text{SSIM}} + \beta_{\text{Edge}} \mathcal{L}_{\text{Edge}} + \beta_{\text{G}} \mathcal{L}_{\text{G}} + \beta_{\text{S}} \mathcal{L}_{\text{S}} + \beta_{\text{Sm}} \mathcal{L}_{\text{Sm}} + \beta_{\text{O}} \mathcal{L}_{\text{O}}. \quad (1)$$

1.6 6. Optimize Gaussian Parameters and Weights (Gradient-Based)

1. Initialize parameters $\{\boldsymbol{\mu}_i, \ell_i, \mathbf{q}_i, a_i\}_{i=1}^N$ and b (using importance sampling from $m(\mathbf{x})$).
2. Use Adam optimizer:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{Total}}, \quad \theta \in \{\boldsymbol{\mu}, \ell, \mathbf{q}, \mathbf{a}, b\}.$$

3. Backpropagate through the Gaussian basis functions via automatic differentiation.

1.7 7. Iterative Training Loop (Proposal-style)

1. **Sample voxel data:** Randomly sample a batch of voxel coordinates (half uniform, half importance-weighted by $m(\mathbf{x})$).
2. **Forward pass:** Compute predictions $\hat{v}_k = f(\mathbf{x}_k)$.
3. **Compute loss:** Evaluate $\mathcal{L}_{\text{Total}}$ from Eq. (1).
4. **Backpropagation:** Compute gradients $\nabla_{\theta} \mathcal{L}_{\text{Total}}$.
5. **Update parameters:** Apply Adam update to all Gaussian parameters and entropy-model parameters.

1.8 8. Densification and Pruning (Adaptive Gaussian Allocation)

1. Accumulate gradient statistics for each Gaussian mean:

$$g_i \leftarrow \|\nabla_{\boldsymbol{\mu}_i} \mathcal{L}_{\text{Total}}\|_2.$$

2. Every K iterations (densify interval), apply:
 - **Split:** if g_i is large and the Gaussian is too large, split it into multiple smaller Gaussians.
 - **Clone:** if g_i is large but the Gaussian is already small, clone to increase local capacity.
 - **Prune:** remove Gaussians with small amplitude $|a_i| < \tau_a$ or excessively large scale.
3. Continue training with updated N .

1.9 9. Stopping Criterion and Output

1. Stop after a fixed number of iterations, or apply early stopping based on a validation distortion metric.
2. Output the trained Gaussian parameters $\{\boldsymbol{\mu}_i, \ell_i, \mathbf{q}_i, a_i\}_{i=1}^N$, bias b , and entropy-model parameters for compression.

2 Objective Function and Regularization

2.1 Implicit volume representation

We represent a normalized volumetric signal $V : \Omega \subset \mathbb{R}^3 \rightarrow \mathbb{R}$ using a sparse mixture of anisotropic 3D Gaussian basis functions. In the notation of the proposals (Step 2), the implicit function is

$$f(\mathbf{x}) = \sum_{i=1}^N w_i G_i(\mathbf{x}; \mathbf{u}_i, \Sigma_i), \quad \mathbf{x} \in \Omega, \quad (2)$$

where the Gaussian basis is

$$G_i(\mathbf{x}; \mathbf{u}_i, \Sigma_i) = \exp\left(-\frac{1}{2} (\mathbf{x} - \mathbf{u}_i)^\top \Sigma_i^{-1} (\mathbf{x} - \mathbf{u}_i)\right). \quad (3)$$

Mapping to our code. Our implementation uses the same functional form but with parameters re-parameterized for stable optimization:

- Mean $\mathbf{u}_i \leftrightarrow \boldsymbol{\mu}_i \in \mathbb{R}^3$ (parameter `model.mu`).
- Covariance Σ_i is represented via a rotation $R(\mathbf{q}_i) \in SO(3)$ and axis scales $\mathbf{s}_i \in \mathbb{R}_+^3$:

$$\Sigma_i = R(\mathbf{q}_i) \text{diag}(\mathbf{s}_i^2) R(\mathbf{q}_i)^\top, \quad \mathbf{s}_i = \exp(\boldsymbol{\ell}_i), \quad (4)$$

where $\boldsymbol{\ell}_i \in \mathbb{R}^3$ are log-scales (`model.log-s`) and $\mathbf{q}_i \in \mathbb{R}^4$ is a quaternion (`model.q`) normalized to unit length.

- Mixture weight $w_i \leftrightarrow a_i$ (amplitude, `model.a`) and a global bias b (`model.b`):

$$f(\mathbf{x}) = \sum_{i=1}^N a_i \exp\left(-\frac{1}{2} \|D_i^{-1} R(\mathbf{q}_i)^\top (\mathbf{x} - \boldsymbol{\mu}_i)\|_2^2\right) + b, \quad (5)$$

with $D_i = \text{diag}(\mathbf{s}_i)$.

2.2 Data fidelity term

Proposal formulation (Step 3). The proposals define the reconstruction loss as mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{M} \sum_{k=1}^M (f(\mathbf{x}_k) - v_k)^2, \quad (6)$$

where $\{(\mathbf{x}_k, v_k)\}_{k=1}^M$ are sampled voxel coordinates and their ground-truth intensities.

Our implementation. We replace pure MSE with a (weighted) Charbonnier penalty to increase robustness to outliers and heavy-tailed noise typical in microscopy (shot noise, sparse bright structures):

$$\mathcal{L}_D = \frac{1}{M} \sum_{k=1}^M \omega(\mathbf{x}_k) \rho(f(\mathbf{x}_k) - v_k), \quad (7)$$

$$\rho(r) = \sqrt{r^2 + \epsilon^2}, \quad \omega(\mathbf{x}) = 1 + \kappa m(\mathbf{x}). \quad (8)$$

Here $m(\mathbf{x}) \in [0, 1]$ is a neurite/edge importance map (constructed from DoG + gradient magnitude), and κ (named `kappa` in the config) emphasizes geometrically important regions (neurites, boundaries). This makes the training geometry-aware without requiring explicit segmentation labels.

Comparison to the proposals. Equation (7) is a strict generalization of (6):

- If $\omega(\mathbf{x}) \equiv 1$ and $\rho(r) = r^2$, we recover the proposal MSE.
- Our choice $\rho(r) = \sqrt{r^2 + \epsilon^2}$ behaves like $|r|$ for large residuals, preventing a small number of extreme voxels from dominating early training.
- The weight $\omega(\mathbf{x})$ explicitly prioritizes geometric correctness (edges / filamentary structures).

2.3 Rate term (compression-aware training)

To train a *codec*, we optimize a rate–distortion objective by adding an entropy-based rate proxy. We quantize parameters using uniform scalar steps Δ (straight-through estimator for gradients):

$$\hat{\theta} = Q(\theta) \approx \text{round}\left(\frac{\theta}{\Delta}\right), \quad (9)$$

and estimate the expected code length (in bits) by a Laplace entropy model:

$$\mathcal{R}(\hat{\theta}) = \mathbb{E}[-\log_2 p_{\text{Lap}}(\hat{\theta})]. \quad (10)$$

In our code, we sum per-parameter groups:

$$\mathcal{L}_{\text{R}} = \mathcal{R}(\hat{\boldsymbol{\mu}}) + \mathcal{R}(\hat{\boldsymbol{\ell}}) + \mathcal{R}(\hat{\mathbf{q}}) + \mathcal{R}(\hat{\mathbf{a}}) + \mathcal{R}(\hat{\mathbf{b}}). \quad (11)$$

This term is not present in the proposals because they describe only the fitting problem, not compression. In a codec setting, $\mathcal{L}_{\text{R}}$ is essential to trade off fidelity vs. bitrate.

2.4 Regularization terms

The proposals (Step 6) list three regularizers: sparsity, overlap, and smoothness. Our implementation contains counterparts, but some are refined to better match 3D anisotropic Gaussians and geometric priorities.

2.4.1 Sparsity regularization

Proposal.

$$\mathcal{L}_{\text{sparsity}}^{(\text{shot})} = \lambda_w \sum_{i=1}^N |w_i|. \quad (12)$$

Our code. We apply the same principle to amplitudes a_i :

$$\mathcal{L}_{\text{S}} = \frac{1}{N} \sum_{i=1}^N |a_i|. \quad (13)$$

Reason (geometry + compression). ℓ_1 sparsity encourages many Gaussians to become negligible and later removable by pruning (e.g., `min_amplitude`), directly improving compression (fewer primitives) and helping geometric correctness by preventing diffuse background “fog”.

2.4.2 Overlap regularization

Proposal. The proposals describe an abstract pairwise overlap penalty:

$$\mathcal{L}_{\text{overlap}}^{(\text{shot})} = \lambda_o \sum_{i,j} \text{overlap}(G_i, G_j), \quad (14)$$

without specifying the exact overlap operator.

Our code (orientation-aware overlap). We instantiate overlap via a Mahalanobis distance under the combined covariance ($\Sigma_i + \Sigma_j$):

$$d_{ij}^2 = (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j)^\top (\Sigma_i + \Sigma_j + \epsilon I)^{-1} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j), \quad (15)$$

$$\text{overlap}_{ij} = \exp(-\tfrac{1}{2}d_{ij}^2), \quad \mathcal{L}_O = \mathbb{E}_{(i,j) \sim \mathcal{P}} [\text{overlap}_{ij}], \quad (16)$$

where $\mathcal{P}$ is a sampled set of random pairs for efficiency.

Comparison to the proposals. Equation (16) is a concrete, geometry-consistent realization of (14):

- It accounts for *anisotropy and orientation* (through Σ_i), which is critical for tubular structures (neurites, vessels).
- It penalizes redundant primitives that occupy the same spatial support, improving identifiability and geometric crispness.
- A simpler isotropic proxy (e.g., $\exp(-\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|/(r_i + r_j))$) ignores orientation and is less faithful to true Gaussian overlap in 3D.

2.4.3 Smoothness regularization: proposal vs. our implementation

Proposal. The proposals state:

$$\mathcal{L}_{\text{smooth}}^{(\text{shot})} = \lambda_s \sum_{i=1}^N \|\nabla_{\mathbf{u}_i} G_i\|_2^2. \quad (17)$$

This is a *Gaussian-parameter* smoothness written in terms of the gradient of each basis w.r.t. its mean $\mathbf{u}_i$.

Our code contains two different “smoothness” notions.

1. **Parameter conditioning (named `smoothness_loss(model)`).** We regularize the covariance parameters to keep primitives well-conditioned:

$$\mathcal{L}_{\text{Sm}} = \underbrace{\mathbb{E}_i [\max(0, s_{\min} - s_i)]}_{\text{avoid collapse}} + c_a \underbrace{\mathbb{E}_i \left[\max(0, \frac{s_{i,\max}}{s_{i,\min}} - 1) \right]}_{\text{limit extreme anisotropy}} + c_m \underbrace{\mathbb{E}_i [\max(0, s_i - s_{\max})]}_{\text{avoid blow-up}}, \quad (18)$$

where $s_i = \exp(\ell_i)$ are axis scales. This does *not* match (17); it is a stability/conditioning prior for Σ_i .

2. **Field smoothness (named `field_grad_smoothness`).** To align with the proposal intent of “smooth transitions”, we add a reconstruction-field gradient penalty:

$$\mathcal{L}_G = \mathbb{E}_{\mathbf{x} \sim \Omega} [\|\nabla_{\mathbf{x}} f(\mathbf{x})\|_2^2], \quad (19)$$

estimated by autograd on randomly sampled coordinates. This is *closer in spirit* to smoothness in implicit fields (and more directly geometry-aligned), even though its gradient is w.r.t. $\mathbf{x}$ rather than $\mathbf{u}_i$.

Why the proposal smoothness is not identical. $\nabla_{\mathbf{u}_i} G_i$ (proposal) and $\nabla_{\mathbf{x}} f(\mathbf{x})$ (ours) are different objects:

- $\nabla_{\mathbf{u}_i} G_i$ constrains how sensitive each *basis* is to moving its center, i.e., it is a parameter-space smoothness.
- $\nabla_{\mathbf{x}} f(\mathbf{x})$ constrains the reconstructed *field geometry* directly in spatial coordinates, which is typically what we want for geometric correctness (suppressing spurious oscillations/ringing).

Geometric priority recommendation. For geometry-first training, $\mathcal{L}_G$ (field gradient) and/or TV on patches (below) are the most directly geometry-aligned. $\mathcal{L}_{S_m}$ should be kept as a covariance-conditioning prior (prevents degenerate Gaussians), but it should be described separately from proposal “smoothness”.

2.4.4 Total variation (TV) on reconstructed patches

We also use 3D total variation on predicted patches:

$$\mathcal{L}_{TV} = \mathbb{E}_{\mathcal{P}} \left[\|\nabla_x f\|_1 + \|\nabla_y f\|_1 + \|\nabla_z f\|_1 \right], \quad (20)$$

computed on local subvolumes for efficiency. TV encourages piecewise-smooth reconstructions while preserving sharp boundaries, complementing the Charbonnier data term for geometric fidelity.

2.4.5 SSIM (structure-preserving regularizer)

We include a 3D SSIM loss on patches:

$$\mathcal{L}_{SSIM} = 1 - \text{SSIM}(f(\mathcal{P}), V(\mathcal{P})), \quad (21)$$

where SSIM is computed from local means/variances/covariances within a small 3D window. While not in the proposals, SSIM stabilizes perceptual/structural similarity and reduces over-smoothing that can occur with TV alone.

2.4.6 Edge-aware loss (geometry-weighted fidelity)

We apply an edge-weighted absolute error on patches:

$$\mathcal{L}_{\text{Edge}} = \mathbb{E}_{\mathbf{x} \in \mathcal{P}} \left[w_{\text{edge}}(\mathbf{x}) |f(\mathbf{x}) - V(\mathbf{x})| \right], \quad (22)$$

with w_{edge} constructed from finite differences of the target patch plus the neurite map. This explicitly prioritizes boundary alignment and filament continuity, which is central for geometric correctness in dendritic/vascular volumes.

2.4.7 Topology-biased patch loss (optional)

We include a lightweight topology-biased loss by mapping predictions to a soft occupancy via a sigmoid threshold and penalizing high-frequency artifacts (TV + high-frequency residual) on that occupancy:

$$\mathcal{L}_T = \text{TV}(\sigma(\gamma(f - \tau))) + c_h \|\sigma(\gamma(f - \tau)) - \text{LPF}(\sigma(\gamma(f - \tau)))\|_1. \quad (23)$$

This is not in the proposals, but it is used to bias the solution toward coherent neurite-like occupancy patterns when training from sparse samples.

2.5 Final objective (rate-distortion with geometric priors)

Combining all terms, the full training objective in our code is:

$$\begin{aligned} \min_{\{\mu_i, \ell_i, q_i, a_i\}, b} \mathcal{L}_{\text{Total}} = & \mathcal{L}_D + \lambda \mathcal{L}_R + \alpha \mathcal{L}_T + \beta_{TV} \mathcal{L}_{TV} + \beta_{SSIM} \mathcal{L}_{SSIM} + \beta_{\text{Edge}} \mathcal{L}_{\text{Edge}} \\ & + \beta_G \mathcal{L}_G + \beta_S \mathcal{L}_S + \beta_{S_m} \mathcal{L}_{S_m} + \beta_O \mathcal{L}_O. \end{aligned} \quad (24)$$

2.6 Direct comparison to the proposals (what matches, what differs)

- **Implicit function:** Matches the proposal form (Eq. (2)–(3)) but uses a rotation+log-scale parameterization for Σ_i (Eq. (4)) and includes a bias b .
- **Data loss:** Proposal uses MSE (Eq. (6)); we use weighted Charbonnier (Eq. (7)) to prioritize neurites/edges and improve robustness.
- **Sparsity:** Same idea as proposal ℓ_1 on weights, applied to amplitudes a_i (Eq. (13)).
- **Overlap:** Proposal leaves overlap unspecified (Eq. (14)); we implement an anisotropy- and orientation-aware overlap via Mahalanobis distance (Eq. (16)), which is more geometrically correct for tubular primitives.
- **Smoothness:** Proposal smoothness is written as $\sum \|\nabla_{u_i} G_i\|^2$ (Eq. (17)). Our `smoothness_loss(model)` is instead a covariance-conditioning prior (Eq. (18)). For geometry-aligned smoothness, our `field_grad_smoothness` (Eq. (19)) and TV (Eq. (20)) are closer to the proposal intent of “smooth transitions” in the reconstructed volume.